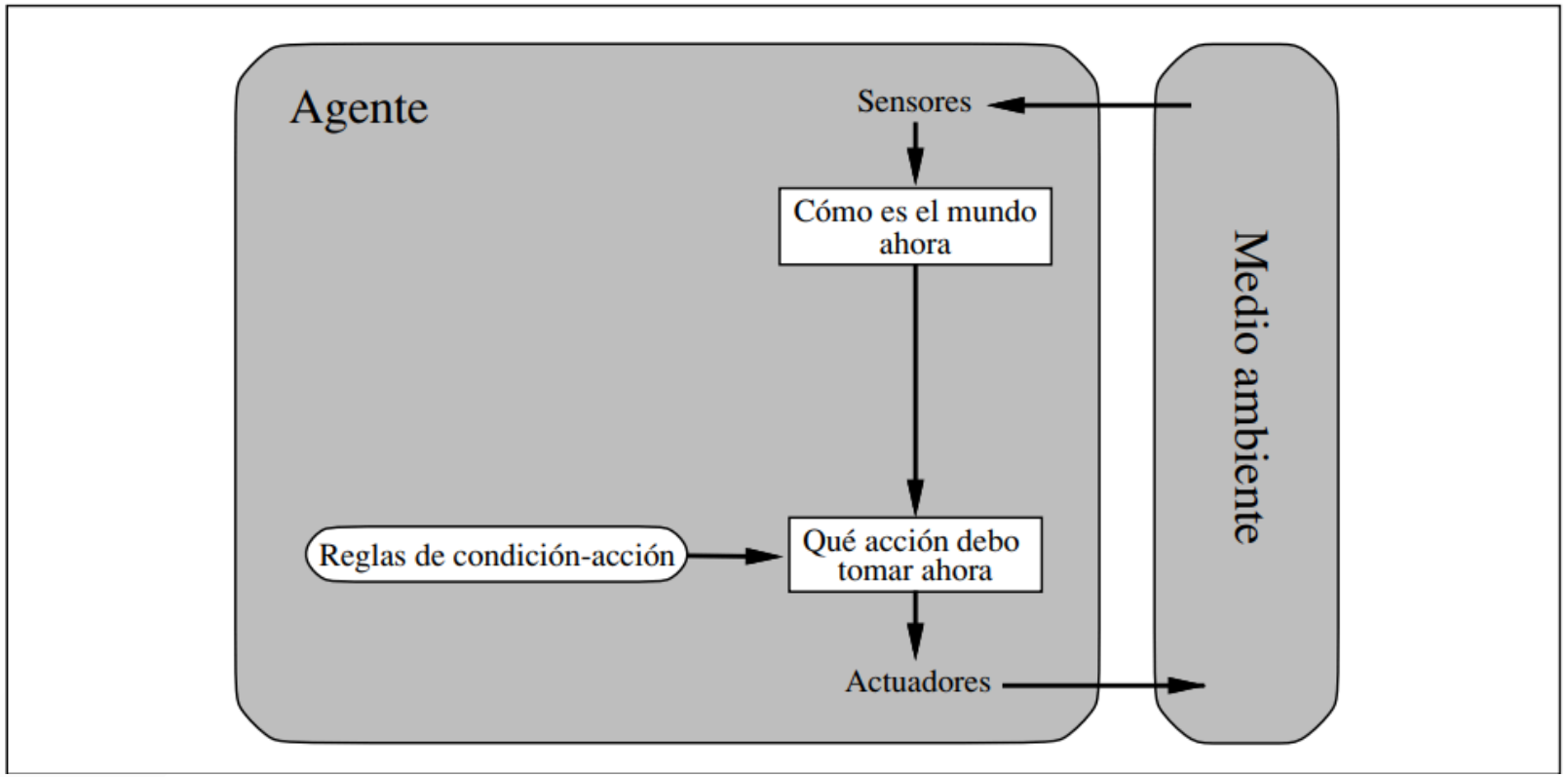
Arquitectura de agentes

Agente esta formado por:

- Arquitectura del agente: Sensores, actuadores y si es físico, hardware para procesar el control.
- Control del agente: El sistema que controla el agente, normalmente software pero podría ser hardware (FPGA, circuito lógico integrado)

Tipos de agentes según su control

Agente reactivo Simple



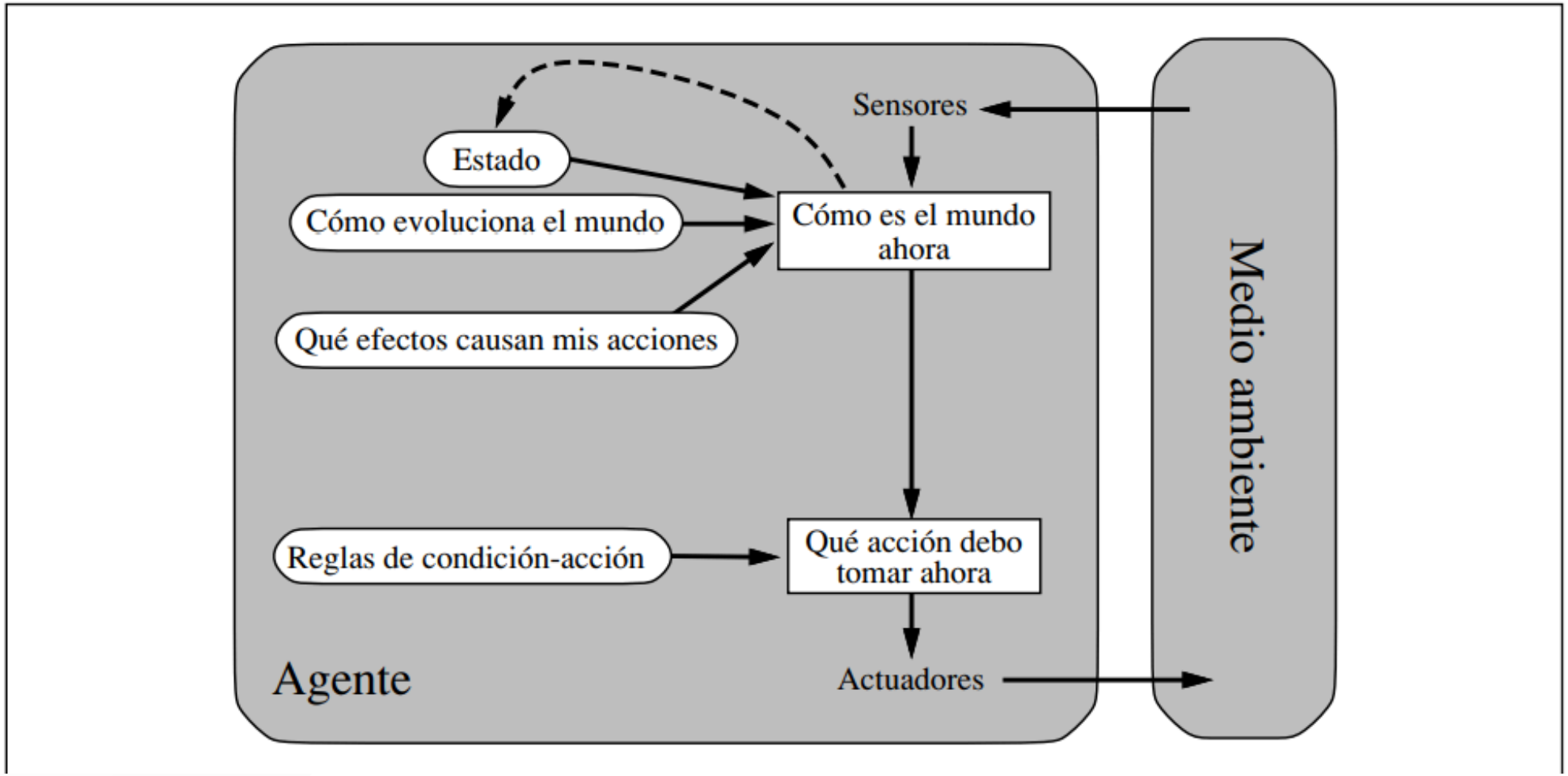
Agente reactivo Simple

- Formalismos: Tabla, if-else, interprete de reglas básicas.

```
función AGENTE-ASPIRADORA-REACTIVO([localización, estado]) devuelve una acción
  si estado  Sucio entonces devolver Aspirar
  de otra forma, si localización  A entonces devolver Derecha
  de otra forma, si localización  B entonces devolver Izquierda
```

- Decisiones sólo en base a la percepción, sin ningún histórico.
- Humanos tienen este tipo de respuestas reactivas => respuestas rápidas a eventos repentinos.
- Simples pero muy limitados, propenso a bucles infinitos (salida usando random)

Agente reactivo basado en modelo



Agente reactivo basado en modelo

Formalismos: Máquina de estados finito, a pila, árboles de comportamiento

```
función AGENTE-REACTIVO-CON-ESTADO(percepción) devuelve una acción
  estático: estado, una descripción actual del estado del mundo
  reglas, un conjunto de reglas condición-acción
  acción, la acción más reciente, inicialmente ninguna

  estado <= ACTUALIZAR-ESTADO(estado, acción, percepción)
  regla <= REGLA-COINCIDENCIA(estado, reglas)
  acción <= REGLA-ACCIÓN[regla]
  devolver acción
```

Agente reactivo basado en modelo

- Mantienen estado interno que depende de la historia de percepción.
- El estado debe actualizarse de alguna forma.
- Guardar la información parcial que no percibes o que pueda ser obsoleta
- La meta en estos agentes es intrínseca y no cambia (Voy a limpiar)

Ejercicio:

Maquina de estados que modela el comportamiento de un Goomba.

Deambula por el mundo de forma psudo-aleatoria. Si encuentra una pared o se sale de los limites establecidos, gira 180º y se muve durante un rato en dirección contraria al obstaculo. Despues vuelve a deambular. Si detecta al jugador se sorprende, una vez terminada la animación de sorpresa le persigue. Si el jugador salta sobre él muere.



Ejercicio:

Maquina de estados que simula el comportamiento de un Agente en un fps.

El agente busca por el mapa a un adversario para dispararle. Cuando lo ve dispara al adversario con su arma cargada. Si el arma no tiene el alcance suficiente cambia a un arma más apropiada para el disparo. Si le queda poca vida intenta buscar un botiquín cercano abandonando cualquier acción que estuviera haciendo en ese momento (excepto cambiar de arma que es bloqueante). Si no hubiese ningún botiquín a la vista, el agente va hacia el adversario para atacarle cuerpo a cuerpo.

¿Cómo se implementa una máquina de estados?

- Switch case
- Con POO
- Dirigida por datos

Variantes:

- Maquina de estados jerárquica
- Maquinas de estado probabilísticas

Behavior Tree

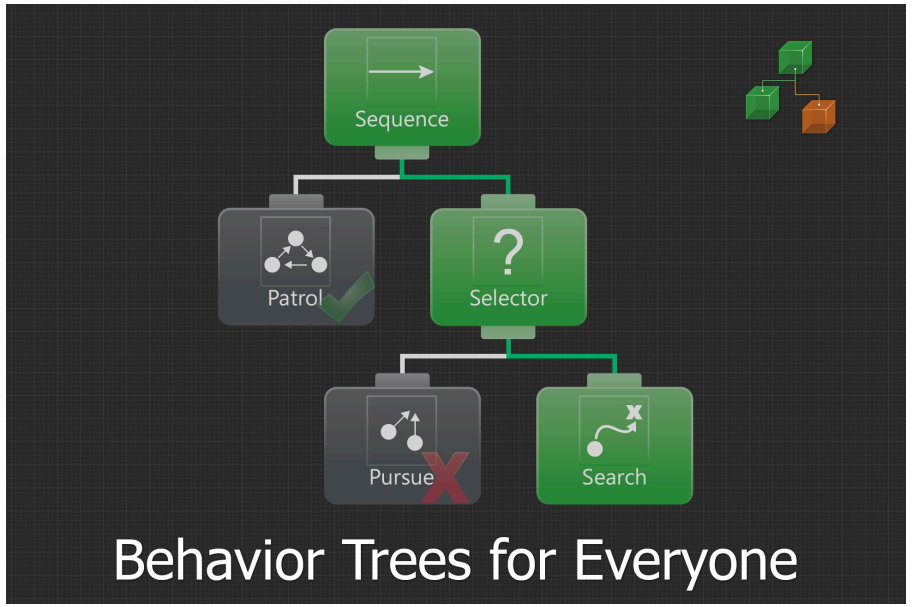
Ejecuta un árbol de comportamiento desde la raíz hasta las hojas de arriba a abajo ude izquierda a derecha. Los nodos no hoja son nodos de decisión, los nodos hoja o terminales son nodos de acción.
Todosl os nodos peude terminar en dos estados: **success o failure**

Los nodos de decisión pueden ser de diferentes tipos:

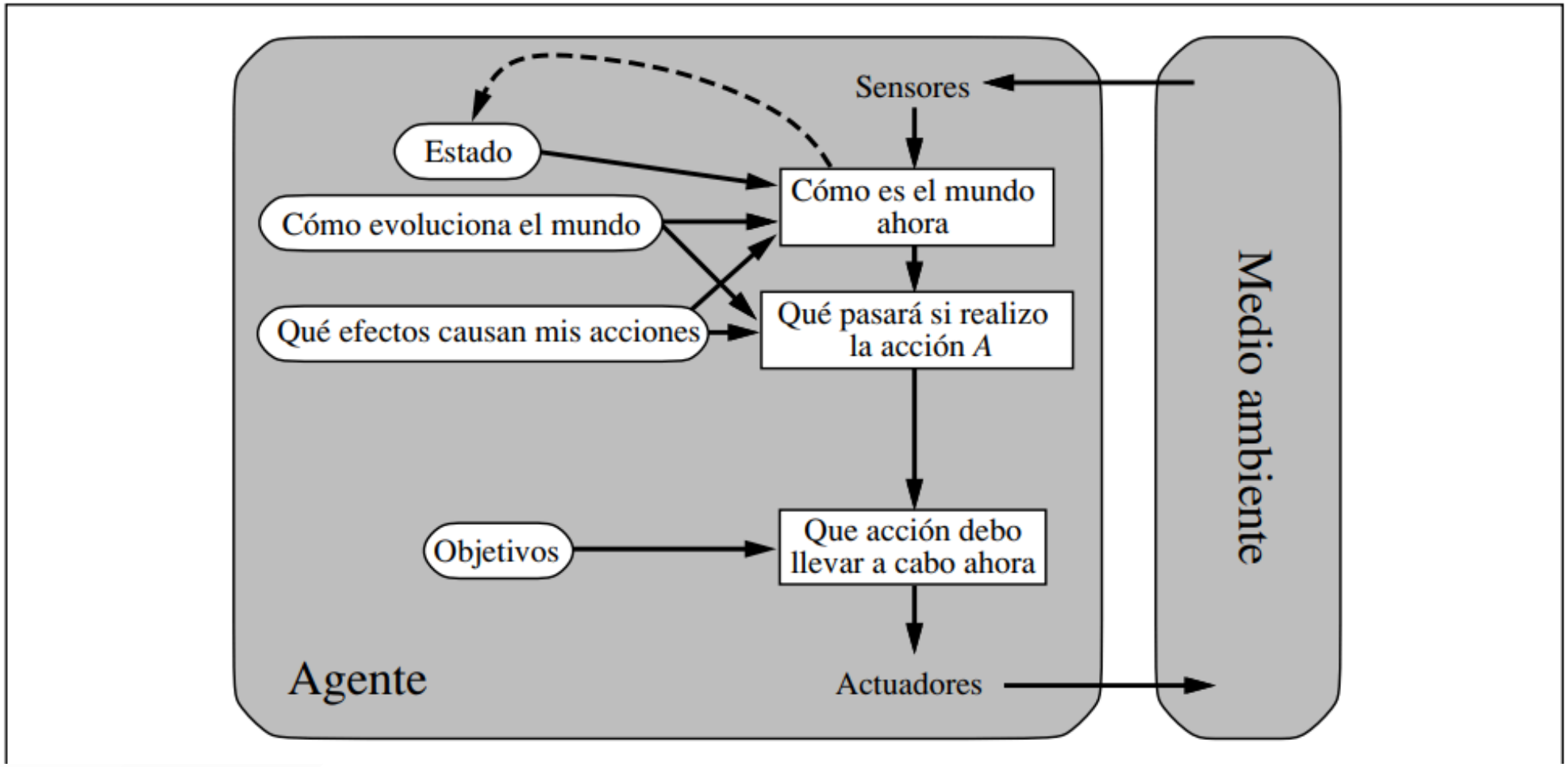
- secuencia: ejecuta todos los nodos uno tras otro mientras sus estados de terminación sean success.
- selector: ejecuta todos los nodos uno tras otro mientras sus estados sean failure. Termina ocn el primer hijo success.

- Parallel: ejecuta n nodos sin tener en cuenta su estad ode terminación
- ETC: Se pueden definir nodos como queramos.

Los nodos temrinales puede ser acciones o condiciones que si se cumplen devuelven success y si no failure.



Agentes basados en objetivos

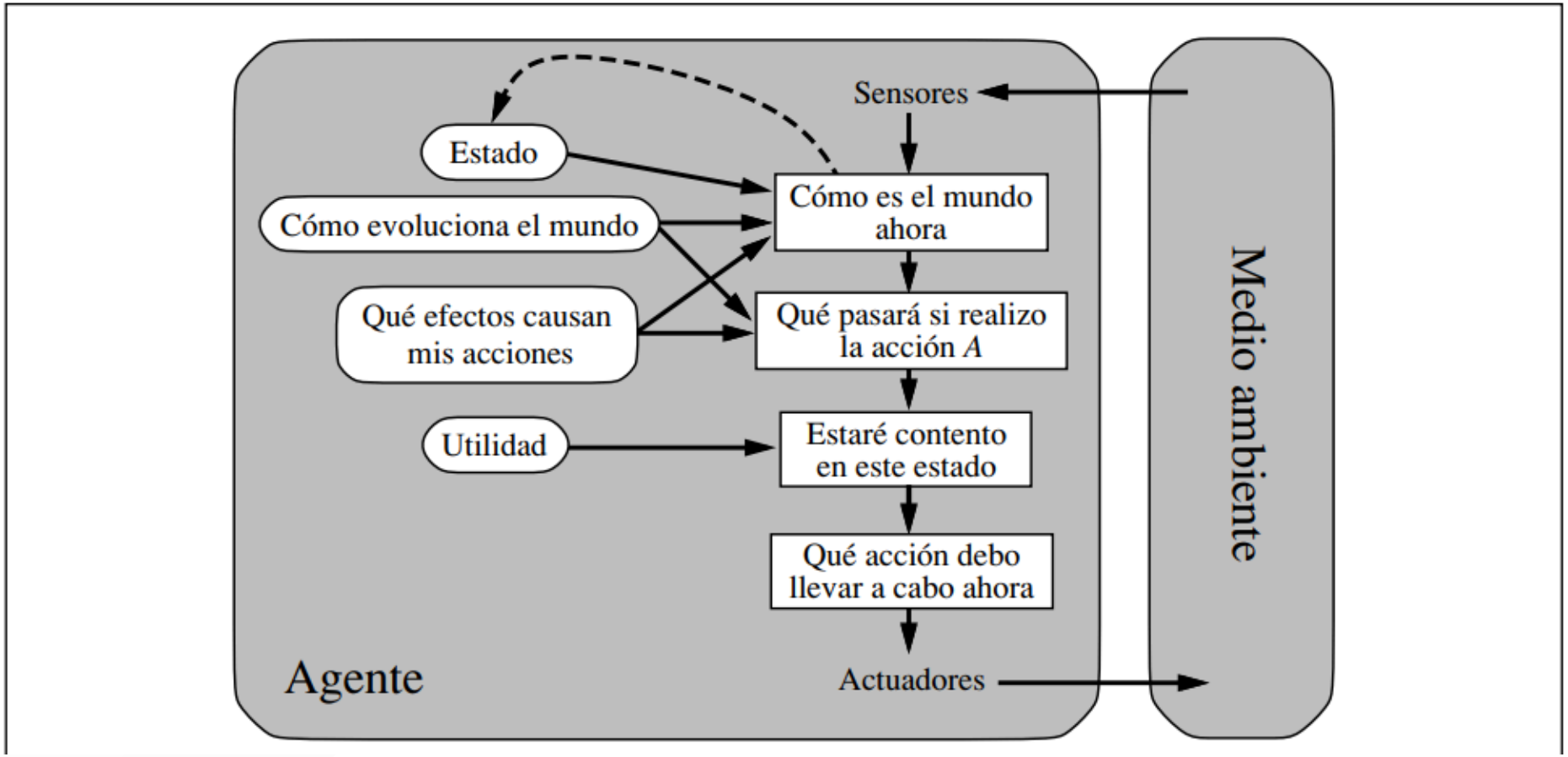


Agentes basados en objetivos

Formalismos: Búsqueda, Planificación etc.

- **Las metas pueden cambiar** por lo que se requiere replanificar.
- Al no existir una meta implícita, una misma decisión será o no acertada en función de la meta.

Agentes basados en utilidad

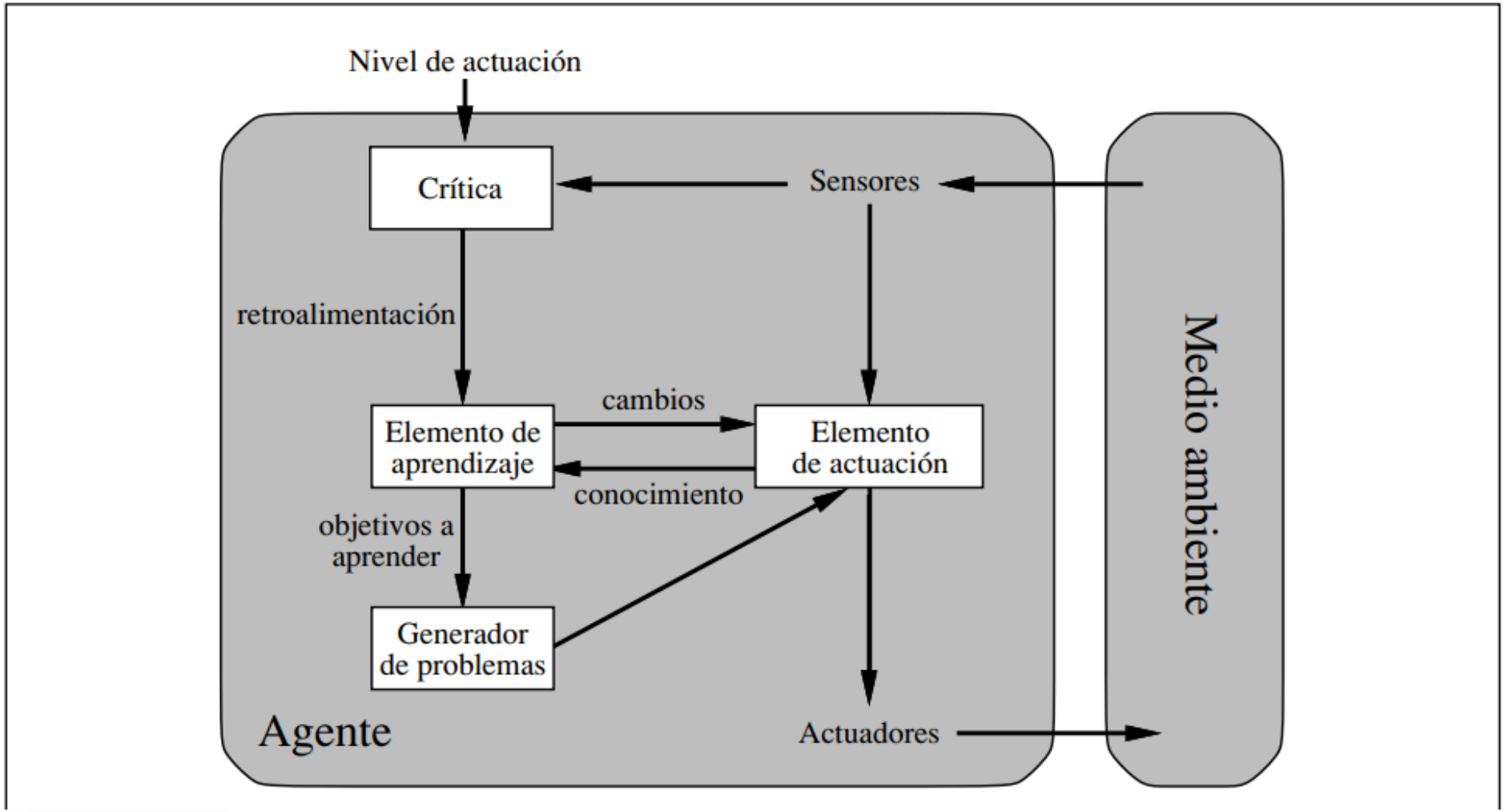


Agentes basados en utilidad

Formalismos: Función de utilidad

- Permite elegir mejor entre diferentes opciones para llegar a un objetivo.
- Permite decidir entre objetivos contradictorios (Llegar rápido o llegar de forma segura)
- Si no tenemos certezas, la función de utilidad permite establecer probabilidades.

Agentes que aprenden



Agentes que aprenden

Formalismos: Aprendizaje por refuerzo, supervisado y no supervisado

- Permite al agente moverse por entornos desconocidos.
- Permite a un mismo agente resolver diferentes problemas
- Elemento de actuación es cualquiera de los agentes anteriores
- La critica observa si lo que se está haciendo es correcto
- Elemento de aprendizaje formula nuevas reglas y actualiza la actuación
- Generador de problemas debe investigar como mejorar

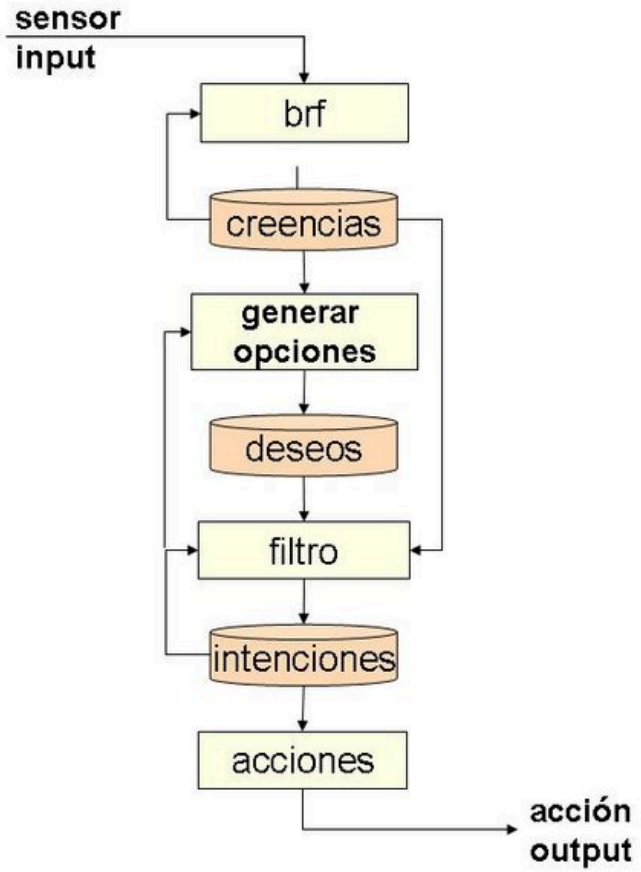
Agentes deliberativos basados en objetivos

Hay varias formas de implementar un agente basado en objetivos:

- Usando reglas de inferencia (Lo veremos más adelante)
- Usando búsqueda y planificación (El siguiente tema :))
- Sistema intencionales usando la arquitectura **BDI** (Belief, Desire, Intention) (Bratman, 1987) (Georgeff, et al., 1998)
  - **Creencias (Beliefs)**: Lo que el agente conoce.
  - **Deseos (Desires)**: Son las metas
  - **Intenciones (Intentions)**: La deliberación del agente actual, plan

Arquitectura BDI

- BRF: función de revisión de creencias
- Función generadora de opciones
- Un filtrado de opciones para quedarnos con la intención actual



Fuente: Ponce, Julio et al. (2014). Inteligencia Artificial.

Agentes vs Objetos

¿Por que usar agentes y no objetos?

- Los objetos son pasivos ya que no actúa hasta recibir un mensaje.
- Los objetos no toman decisiones
- Los objetos obedecen órdenes
- POO es una solución adecuada par entornos conocidos y bien definidos.

Se puede utilizar un paradigma de orientación a objetos para implementar uno de agentes

Sistemas multi-agentes

Sistema Multi-agente (MAS) es un conjunto de agentes autónomos, normalmente heterogéneos que trabajan en resolver un problema entre todos ellos.

- Inteligencia artificial distribuida. (DAI)
- Necesitan comunicarse para resolver los problemas
- Pueden tener información parcial y redundancia de información

Aunque lo más normal es que el agente sea autónomo y tome sus propias decisiones, se puede tener una coordinación centralizada

Ventajas de los SMA

- Normalmente aumenta la potencia del sistema
- Robustez y tolerancia a fallos
- Sinergias entre agentes (Comportamiento emergente)
- No es necesario que el agente conozca toda la información (Perfiles de seguridad)
- Se adapta mejor a entornos dinámicos

Interacción entre los agentes

Los agentes deben interactuar entre si en base a tres parámetros:

- Objetivos de los agentes que interactúan: compatibles / incompatibles
- Recursos disponibles para realizar las tareas: suficientes / insuficientes
- Habilidades del agente para realizar la tarea: suficientes / insuficientes

Tipos de colaboración

Tipo	Objetivos	Recursos	Habilidad
Independencia	Compat	Suf	Suf
Colaboración simple	Compat	Suf	Insuf
Obstrucción	Compat	Insuf	Suf
Colaboración coordinada	Compat	Insuf	Insuf
Competición individual	Incompat	Suf	Suf
Competición por equipos	Incompat	Suf	Insuf
Conflicto individual	Incompat	Insuf	Suf
Conflicto colectivo	Incompat	Insuf	Insuf

Ejemplos

- Los jugadores del Mus
- Brokers online
- Drones fireworks
- Coches autónomos
- Agentes de compra online

Organización de los agentes

Es fundamental en las sociedades de agentes. Se debe definir el número y tipo de agentes, el rol de cada agente y sus relaciones.

- Organización definida por el programador
- Organización emergentes como resultado de la interactividad
- Organización jerárquica
- Organización igualitaria

Las relaciones pueden ser **estáticas** o **dinámicas**.

Tipos de relaciones entre agentes

- Conocimiento: A conoce a B y puede comunicarse con B
- Comunicación: Existe un canal de comunicación entre A y B
- Subordinación: B realiza una tarea que A le pide:
  - Si no se puede negar estática (Maestro / Esclavo)
  - Si se puede negar o negociar dinámica
- Operativa: A depende de que B realice antes otra tarea
- De información: A cree a B
- Conflicto: Varios agentes necesitan mismos recursos
- Competición: Agentes con objetivos incompatibles

Cooperación

Varias formas de cooperar:

- Cooperación intencional: los agentes son conscientes de que deben colaborar
- Cooperación desde el punto de vista del observador (los agentes no son conscientes)

La cooperación es útil para mejorar el rendimiento del grupo y resolver conflictos.

Tipos de cooperación

- Agrupamiento de agentes: Biología
- Compartir información
- Especialización de tareas
- Compartir recursos
- Resolución de conflictos mediante arbitraje

La comunicación

Es la base de la colaboración y la coordinación. Los agentes se comunican para solicitar tareas, recursos, proponer información, compartir creencias o coordinarse para realizar una tarea.

- **Señal**: modificación del entorno. No tiene destinatario fijo
- **Mensaje**: el agente envía el mensaje a otro agente. Tiene un destinatario
  - Síncronos: Se espera a que el receptor conteste inmediatamente
  - Asíncrono: No se espera la recepción del mensaje y se retarda en el tiempo

Tipos de colaboración

- Reparto de tareas:
  - Distribuida: cada agente gestiona los recursos y servicios
  - Centralizada: Un agente coordinador se encarga del reparto
  - Emergente: El reparto surge de las interacciones.
- Contrato: el proveedor se compromete a realizar una tarea y el cliente a no encargársela a otro. Los contratos pasan por fases (asignado, cerrado, hecho, roto)
  - **Smart contracts**: contratos que realizan agentes software que tiene validez. Están muy extendidos últimamente y se apoyan en **blockchain**.

- Red de contratos:
  - Se publica una tarea
  - Contratistas envían ofertas (red de conocidos, registro, memoria)
  - Cliente evalúa las ofertas y se queda con la que más le interesa
  - Se formaliza el contrato
- Cosas a especificar en un contrato:
  - Calidad mínima requerida
  - Como se evaluará
  - Fecha límite de realización

Se necesita un protocolo común de comunicación

Para que se entiendan correctamente dos máquinas, ambas deben hablar el mismo lenguaje.

Para ello se crearon diferentes estandares. La FIPA (1996-2005) creo el denominado Standar FIPA con el objetivo de construir agentes interoperables.
El estándar solo fija el comportamiento externo de los agentes: **ACL framework for agents**
Actualmente está integrado en la IEEE Computer Society